Java OOP — Part 3.1: Foundations and Object Creation

Object-oriented programming models software as **objects that own their data and the operations on it**. Part 3.1 builds the base for everything else in the OOP track: what classes and objects are, how Java organizes them, how an object is created in memory, and how constructors and initialization work.

3.1.1 Paradigms, Classes and Objects

Programming paradigms

A **paradigm** is a style of structuring programs: a way of deciding what the basic building blocks are and how they cooperate. Most languages support more than one.

Paradigm	Core idea	Building block	Example languages
Procedural	A program is a sequence of steps grouped into procedures that operate on shared data	Function	C, Pascal
Object-oriented	A program is a set of objects that hold their own data and talk by sending messages (method calls)	Object (instance of a class)	Java, C#, C++, Python
Functional	A program is a composition of functions with no hidden state, ideally without side effects	Function as a value	Haskell, Scala, Clojure
Declarative	You state *what* result you want, not the steps	Query, rule, expression	SQL, HTML, Prolog

Java is **class-based and object-oriented** at its core. Since Java 8 it also supports functional style (lambdas, streams, `Optional`), and since Java 16 to 21 it supports data-oriented style (records, sealed types, pattern matching). Real Java code mixes these.

The problem OOP solves

In **procedural** code, data and the functions that work on it are separate. Any function can touch any data, so a change to a data layout ripples through every function that uses it.

```
// Procedural style in Java (the style OOP replaces)
static double[] balances = new double[100];           // global-ish data
static void withdraw(int acct, double amt) { balances[acct] -= amt; }   //
anyone can call this
static void deposit(int acct, double amt)  { balances[acct] += amt; }
// Nothing prevents: balances[5] = -1_000_000;  from anywhere in the program
```

In **OOP**, data and behavior are bundled and the data is protected:

```
class Account {
    private double balance;                        // hidden state
    void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("amount must be
positive");
        balance += amt;
    }
    boolean withdraw(double amt) {
        if (amt <= 0 || amt > balance) return false;   // rules live next to
the data
        balance -= amt;
        return true;
    }
    double getBalance() { return balance; }
}
```

Benefits OOP aims for:

- **Modularity:** each class is a self-contained unit that can be developed and tested alone.
- **Reusability:** classes and their behavior are reused through composition and inheritance.
- **Maintainability:** a change to the inside of a class does not affect callers that use only its public methods.
- **Extensibility:** new behavior is added by new classes that fit existing abstractions (polymorphism).
- **Safety:** invariants (a balance is never negative) are enforced in one place.

OOP rests on four pillars: **encapsulation, inheritance, polymorphism, abstraction**. This part covers encapsulation in depth; the other three get their own parts.

Honest caveat: Java is *not purely* object-oriented. It has 8 primitive types that are not objects, `static` members that belong to no object, and it allows plain functions-in-disguise (static methods). Interviewers like this question.

Class and object

A **class** is a *blueprint* (a user-defined type) that describes what data its objects hold and what they can do. An **object** is a *concrete instance* of a class that exists in memory at runtime, with its own copy of the data.

Think of a class `House` as the architect's drawing, and each object as a real house built from it. Many houses can be built from one drawing, each with its own paint colour and its own occupants, but they all have the same layout.

Every object has three properties:

Property	Meaning	In Java
State	The data the object currently holds	Values of its instance fields
Behavior	What the object can do	Its methods
Identity	What makes it *this* object, distinct from any other with equal state	Its location in the heap; compared with `==`

```
class Car {
    // state (instance variables)
    String model;
    int speed;

    // behavior (methods)
    void accelerate(int delta) { speed += delta; }
    void brake()               { speed = 0; }
}

class Demo {
    public static void main(String[] args) {
        Car a = new Car();      // object 1
        Car b = new Car();      // object 2: same class, separate state
        a.model = "Swift";
```

```
        b.model = "Creta";
        a.accelerate(40);
        System.out.println(a.speed + " " + b.speed);   // 40 0
        System.out.println(a == b);                    // false: two
identities
    }
}
```

Class vs object

	Class	Object
Nature	Logical template, a type	Physical instance in memory
Created with	`class` keyword (once, at design time)	`new` keyword (any number of times, at runtime)
Memory	Loaded once by the ClassLoader (metadata in Metaspace)	Each one takes heap space for its own fields
Data	Declares fields; static fields exist once	Holds actual values of instance fields
Count	One per type	Zero, one or millions

References. A variable of class type does not hold the object; it holds a **reference** to it. `Car a = new Car();` creates the object on the heap and stores its reference in `a`. Consequences you must know:

```
Car a = new Car();
Car b = a;            // copies the REFERENCE: a and b point to the same
object (aliasing)
b.speed = 99;
System.out.println(a.speed);   // 99

Car c = null;         // a reference that points to nothing
// c.accelerate(5);   // NullPointerException: no object behind the reference
a = null;             // object still reachable via b
b = null;             // now unreachable: eligible for garbage collection
```

Kinds of types you can declare

Kind	Keyword	Purpose (covered in)
Concrete class	`class`	Create objects
Abstract class	`abstract class`	Partial implementation, cannot be instantiated (3.2)
Final class	`final class`	Cannot be extended (3.2)
Interface	`interface`	A contract (3.4)
Enum	`enum`	Fixed set of constants, with behavior (3.5)
Record	`record`	Immutable data carrier (3.5)
Sealed type	`sealed`	Restricted hierarchy (3.5)
Annotation	`@interface`	Metadata
Nested/inner class	declared inside another type	Helper or tightly bound types (3.2)

Structure of a Java program with OOP

A source file follows this order: `package`, `import`s, then type declarations.

```
// File: src/com/example/bank/Account.java
package com.example.bank;                   // 1. where this class lives

import java.util.ArrayList;                 // 2. other classes we use
import java.util.List;

public class Account {                      // 3. the type; file name must
match: Account.java

    // 4. constants and static members
    public static final double MIN_BALANCE = 0.0;

    // 5. instance fields (state), usually private
    private final String id;
    private double balance;
    private final List<String> history = new ArrayList<>();
```

```
    // 6. constructors
    public Account(String id, double opening) {
        this.id = id;
        this.balance = opening;
    }

    // 7. methods (behavior): public API first, private helpers last
    public void deposit(double amt) { balance += amt; history.add("D:" +
amt); }
    public double getBalance()      { return balance; }
    public String getId()           { return id; }

    // 8. nested types, if any, and overrides such as toString / equals /
hashCode
    @Override public String toString() { return id + " [" + balance + "]"; }
}
```

```
// File: src/com/example/bank/Main.java  (a separate 'driver' class holds
main)
package com.example.bank;

public class Main {
    public static void main(String[] args) {
        Account acc = new Account("A-101", 500);
        acc.deposit(250);
        System.out.println(acc);          // A-101 [750.0]
    }
}
```

Rules of file structure

- At most **one `public` top-level type per file**, and the file name must equal its name (`Account.java`). A file may hold any number of non-public top-level types, but one type per file is the norm.
- `package` must be the first statement and must match the directory path (`com/example/bank`). Package names are lowercase, in reverse-domain style, to avoid clashes.
- `import` brings other packages' classes into scope by simple name. `java.lang` (`String`, `Object`, `Math`, …) is imported automatically. `import java.util.*;` imports all types of a package but not its sub-packages.

- `main` is just a static method; it can sit in any class. Keeping it in a small driver class (or a framework such as Spring Boot's application class) separates *starting* the program from *modeling* the domain.

Compile and run with packages

```
# from the project root
javac -d out src/com/example/bank/*.java      # -d out: put .class files in
out/com/example/bank/
java  -cp out com.example.bank.Main          # run by fully-qualified class
name
```

Typical member order inside a class (a convention that most style guides follow): static constants, static fields, instance fields, constructors, public methods, protected and package-private methods, private methods, nested types.

Predefined vs user-defined classes

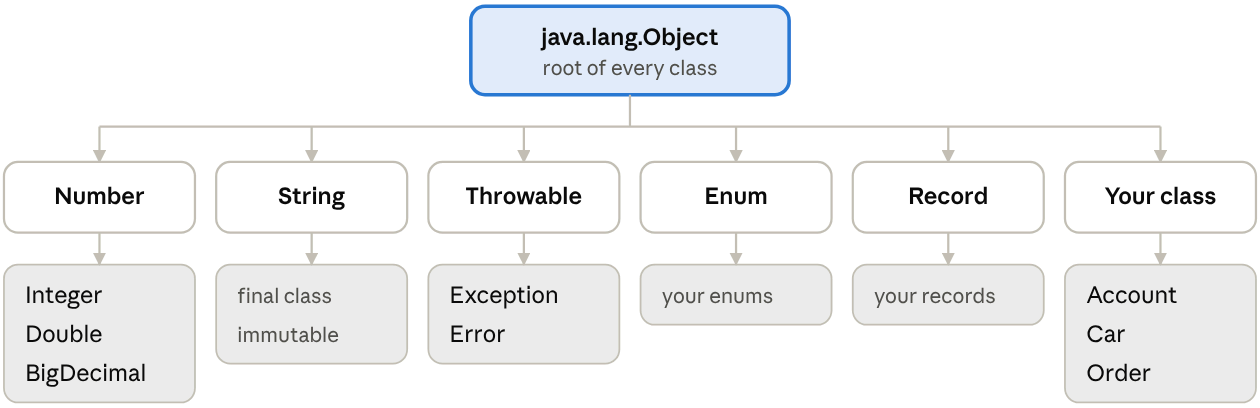
	Predefined (library) classes	User-defined classes
Written by	The JDK authors (shipped in the `java.base` and other modules)	You and your team
Examples	`String`, `Math`, `ArrayList`, `LocalDate`, `Scanner`, `Thread`	`Account`, `OrderService`, `Invoice`
Import	`java.lang` automatic; others need `import`	Same package: none; else import
Role	General building blocks	Model your problem domain

In a typical Spring Boot service, user-defined classes fall into roles: **entity** (mapped to a table), **DTO** (data crossing an API boundary), **service** (business logic), **repository** (data access) and **configuration**. Good OOP design means each class has one clear responsibility.

3.1.2 The Java Class Hierarchy, Object and Class

Java has a **single-root class hierarchy**: every class is a direct or indirect subclass of `java.lang.Object`, and every class has exactly one direct superclass (single inheritance of classes).

Every class in Java descends from Object



Every class extends Object, directly or through a parent. Interfaces such as Comparable sit outside this tree. Boolean, Character, Thread and thousands of other classes also extend Object directly.

Class hierarchy rooted at java.lang.Object

Read the drawing top to bottom: each box extends the one above it. If you write `class Car { }` the compiler treats it as `class Car extends Object { }`.

What the hierarchy gives you

- **A universal type.** A variable of type `Object` can hold a reference to any object, including arrays. Before generics, collections stored `Object`.
- **Guaranteed common behavior.** Every object can be printed (`toString`), compared (`equals`), hashed (`hashCode`), locked (`synchronized`, `wait`, `notify`) and asked for its runtime type (`getClass`).
- **Polymorphism.** Code written against a parent type works with every subclass.

```
Object o1 = "text";          // a String
Object o2 = 42;              // autoboxed Integer
Object o3 = new int[]{1, 2}; // arrays are objects too
Object o4 = new Account("A-1", 10);
```

Walking up the hierarchy at runtime

```
Class<?> c = Integer.class;
while (c != null) {
    System.out.println(c.getName());
    c = c.getSuperclass();
}
// java.lang.Integer
// java.lang.Number
// java.lang.Object
```

`ArrayList` gives `java.util.ArrayList`, `AbstractList`, `AbstractCollection`, `Object`. Interfaces are not in this chain: `getInterfaces()` lists them separately, and `Object.class.getSuperclass()` is `null`.

Notes on the hierarchy

- **Interfaces are outside the class tree.** An interface does not extend `Object`, but you can call `Object`'s methods through an interface reference, because every implementing object is an `Object`.
- **Arrays** extend `Object` and implement `Cloneable` and `Serializable`. `new int[0].getClass().getSuperclass()` is `Object`.
- **Enums** extend `java.lang.Enum`, **records** extend `java.lang.Record`, and **exceptions** extend `Throwable`. You cannot extend those parents by hand.
- **A class cannot extend two classes**, so the extends clause always names one class. Multiple *types* come from interfaces (Part 3.4).
- **Final classes** such as `String` and `Integer` end a branch: no subclasses are allowed (Part 3.2).

java.lang.Object: the root class

`Object` declares 11 methods that every object inherits. Parts 3.3 and 3.5 treat the important ones in depth; this table is the map.

Method	Purpose	Default behavior	Note
`getClass()`	Runtime class of the object	Returns its `Class` object	`final`: cannot be overridden

Method	Purpose	Default behavior	Note
`hashCode()`	Integer used by hash tables	Identity-based (derived from the object's identity)	Override together with `equals`
`equals(Object)`	Logical equality	Same as `==` (same reference)	Override for value types
`toString()`	Text description	`ClassName@hexHashCode`	Override for readable logs
`clone()`	Field-by-field copy	Shallow copy; needs `Cloneable`	`protected`; considered flawed
`finalize()`	Cleanup before collection	Does nothing	Deprecated for removal; never use
`wait()`, `wait(long)`, `wait(long,int)`	Release the monitor and wait	Blocks until notified or timed out	`final`; needs the object's lock
`notify()`	Wake one waiting thread		`final`; needs the object's lock
`notifyAll()`	Wake all waiting threads		`final`; needs the object's lock

What the defaults mean in practice:

```
class Point { int x, y; Point(int x, int y) { this.x = x; this.y = y; } }

Point p = new Point(1, 2);
Point q = new Point(1, 2);
System.out.println(p);              // Point@1b6d3586  (class name + @ + hex
hash: useless)
System.out.println(p.equals(q));    // false  (default equals is identity)
System.out.println(p.equals(p));    // true
System.out.println(p.hashCode() == q.hashCode());   // almost certainly false
System.out.println(p.getClass() == q.getClass());   // true  (same runtime
class)
```

`wait`, `notify` and `notifyAll` live on `Object` because **every object can act as a monitor lock**. This is the link back to Section 1 (the object header's lock bits) and to `synchronized`.

java.lang.Class: the runtime description of a type

For every type the JVM loads (class, interface, enum, record, array, primitive), there is exactly one **`java.lang.Class` object per ClassLoader**. It is the object created during the **Loading** phase you saw in Section 1. It holds the type's name, its parent, its interfaces, modifiers, and handles to its fields, methods and constructors. It lives on the heap; the heavy metadata sits in Metaspace.

You cannot call `new Class(...)`. The JVM creates it. You get a reference in three ways:

```
Class<Account> a = Account.class;                 // 1. class literal
(compile-time, no object needed)
Class<?> b = acc.getClass();                      // 2. from an instance: the
RUNTIME class
Class<?> c = Class.forName("com.example.bank.Account");   // 3. by name
(loads and initializes; may throw ClassNotFoundException)

int.class;  void.class;  int[].class;  String[].class;     // primitives,
void and arrays have Class objects too
System.out.println(a == b && b == c);             // true when all three use
the same ClassLoader
```

What you can ask a `Class` object

```
Class<?> k = ArrayList.class;
k.getName();            // "java.util.ArrayList"   (fully qualified)
k.getSimpleName();      // "ArrayList"
k.getPackageName();     // "java.util"
k.getSuperclass();      // class java.util.AbstractList
k.getInterfaces();      // [List, RandomAccess, Cloneable, Serializable]
k.getModifiers();       // bit flags; decode with
java.lang.reflect.Modifier.toString(...)
k.isInterface();  k.isArray();  k.isEnum();  k.isRecord();  k.isPrimitive();
k.getDeclaredFields();  k.getDeclaredMethods();  k.getDeclaredConstructors();
// reflection
Object o = k.getDeclaredConstructor().newInstance();   // create an object
without writing 'new'
```

Static type vs runtime type

```
Animal a = new Dog();
System.out.println(a.getClass().getSimpleName());    // Dog  (runtime type:
the real object)
// the declared (static) type is Animal; getClass() always reports the real
class
System.out.println(a instanceof Animal);             // true  (instanceof
accepts subtypes)
System.out.println(a.getClass() == Animal.class);    // false (exact class
comparison)
```

That difference is the heart of correct `equals()` implementations in Part 3.5.

Generics are erased at runtime

```
System.out.println(new ArrayList<String>().getClass() == new
ArrayList<Integer>().getClass());  // true
// List<String>.class is illegal: there is only List.class
```

Where `Class` is used in real code. Frameworks such as Spring, Hibernate and Jackson read class metadata through `Class` and reflection to create beans, map entities to tables and (de)serialize JSON. Reflection is powerful but slower than direct calls, bypasses compile-time checks, and since Java 16 and 17 the module system blocks deep reflection into JDK internals unless explicitly opened. Reflection is covered in its own module later.

3.1.3 Encapsulation, Data Hiding and Access Modifiers

Encapsulation means bundling data and the methods that operate on it into one unit (the class) and **controlling access** so that outsiders can use the object only through a deliberately small public interface. It is the first of the four OOP pillars, and the one every other design decision depends on.

Why encapsulate

- **Protect invariants.** A balance must never go negative, an age must be positive, a list must not contain nulls. When fields are private, only the class's own methods can change them, so the rule is enforced in exactly one place.
- **Freedom to change internals.** Callers depend on the public methods, not on how data is stored. You can switch a `double` to `BigDecimal`, or an array to a `Map`, without touching callers.

- **Easier debugging.** If a field has a wrong value, only the class's own methods could have put it there.
- **Controlled concurrency.** Shared mutable state is safe to manage only when access to it goes through a few methods.

```
// NOT encapsulated: anyone can break the rule
class BadAccount {
    public double balance;
}
BadAccount b = new BadAccount();
b.balance = -5_000;                  // legal, and nobody can stop it

// Encapsulated: the rule lives with the data
class GoodAccount {
    private double balance;                      // data hiding

    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException("deposit must be
positive");
        balance += amount;
    }
    public void withdraw(double amount) {
        if (amount <= 0 || amount > balance) throw new
IllegalStateException("invalid withdrawal");
        balance -= amount;
    }
}
```

Data hiding, encapsulation and abstraction are different

These three words are mixed up constantly. Be able to separate them.

	Data hiding	Encapsulation	Abstraction
Meaning	Outsiders cannot touch the data directly	Data and behavior are packaged together, with controlled access	Showing *what* an object does and hiding *how*
Achieved with	`private` fields	Classes plus access modifiers	Interfaces and abstract classes

	Data hiding	Encapsulation	Abstraction
Focus	Security of the data	Organization and protection of the unit	Simplifying the view for the user of the type
Level	Implementation detail	Design of one class	Design of the whole type system

A common one-line summary: **data hiding is the technique, encapsulation is the goal that uses it, abstraction is a different goal that hides complexity.**

Tightly encapsulated classes

A class is **tightly encapsulated** if **every data member (field) in it is `private`**, and, when the class extends another class, **every parent class in the chain is also tightly encapsulated**. Whether getters and setters exist does not matter: the test is only about field visibility.

```
class A { private int x; }                          // tightly encapsulated
class B extends A { private int y; }                // tightly encapsulated
(parent is too)

class C { int x; }                                  // NOT: package-private
field
class D extends C { private int y; }                // NOT: its parent is not
tightly encapsulated

class E { private int a; public int b; }           // NOT: one public field
breaks it
class F extends A { }                               // tightly encapsulated: no
fields, parent is OK
```

Interview answers use this exact definition. In real design, treat it as the minimum standard.

Getters and setters

A **getter** reads a private field; a **setter** changes it, usually after validation. The JavaBeans convention is:

Field	Getter	Setter
`String name`	`getName()`	`setName(String name)`
`boolean active`	`isActive()` (or `getActive()`)	`setActive(boolean active)`

```
public class Employee {
    private String name;
    private int age;

    public String getName() { return name; }
    public void setName(String name) {
        if (name == null || name.isBlank()) throw new
IllegalArgumentException("name required");
        this.name = name.trim();                // setter can validate and
normalize
    }

    public int getAge() { return age; }
    public void setAge(int age) {
        if (age < 18 || age > 70) throw new IllegalArgumentException("age out
of range");
        this.age = age;
    }
}
```

Variations: **read-only** (getter only, field set in the constructor and `final`) is the safest and most common; **write-only** (setter only) is rare, for example a password you accept but never return.

Getters and setters alone are not encapsulation. Auto-generating a getter and setter for every field gives you a public field with extra typing. Real encapsulation exposes **behavior**, not state:

```
// Anemic: caller must know the rules
acc.setBalance(acc.getBalance() - 100);

// Encapsulated: the object knows the rules
acc.withdraw(100);
```

The guideline is "**tell, don't ask**": tell the object what to do instead of pulling its data out and deciding for it.

Leaking mutable internals. Returning a reference to a private mutable object hands outsiders the keys:

```
class Team {
    private final List<String> members = new ArrayList<>();

    public List<String> getMembers() { return members; }     // LEAK: caller
can add, remove, clear

    // Fixes:
    public List<String> getMembersView()  { return
Collections.unmodifiableList(members); }  // read-only view
    public List<String> getMembersCopy()  { return List.copyOf(members); }
// immutable snapshot
    public void addMember(String m)       {
members.add(Objects.requireNonNull(m)); }        // controlled mutation
}
```

The same applies on the way **in**: if a constructor stores the caller's `Date` or array directly, the caller can still modify it later. Make **defensive copies** (Part 3.5, immutability).

Access modifiers

Access modifiers decide **which code may see a class or member**. Java has four levels; the fourth has no keyword.

Modifier	Same class	Same package	Subclass in another package	Anywhere else
`public`	Yes	Yes	Yes	Yes
`protected`	Yes	Yes	Yes (with a condition, below)	No
(none): package-private, "default"	Yes	Yes	No	No
`private`	Yes	No	No	No

Where each modifier may be used

Element	`public`	`protected`	package-private	`private`
Top-level class or interface	Yes	No	Yes	No
Nested class	Yes	Yes	Yes	Yes
Field, method, constructor	Yes	Yes	Yes	Yes
Local variable, parameter	No modifier allowed			
Interface members	Methods and constants are implicitly `public`; nested types are `public static`			`private` methods allowed since Java 9

`private`: visible only inside the **declaring class** (including its nested classes). The restriction is per *class*, not per object, so one object may read another object's private fields:

```
class Money {
    private final long paise;
    Money(long p) { paise = p; }
    boolean isMoreThan(Money other) { return this.paise > other.paise; }   //
legal: same class
}
```

Package-private (default): no keyword; visible to every class in the **same package**. Use it for implementation classes and helpers that other packages should not see; it is the right default for classes that are not part of your API.

`protected`: visible in the same package, **and** to subclasses in other packages, but with a catch. Outside the package, a subclass may access the protected member only through **a reference of its own type (or a subtype)**, not through a reference of the parent type.

```
// File a/Base.java
package a;
public class Base {
    protected void hello() { }
}

// File b/Sub.java
package b;
import a.Base;
public class Sub extends Base {
    void test() {
        hello();                // OK: inherited member of this object
        this.hello();           // OK
        new Sub().hello();      // OK: reference of type Sub
        Base base = new Base();
        // base.hello();        // COMPILE ERROR: reference of the parent
type, from another package
    }
}
```

`public`: visible everywhere that can see the class itself. Everything public is part of your contract: you cannot change it without breaking callers, so expose as little as possible.

Rules and traps

- A **top-level** class can only be `public` or package-private. `private class X` and `protected class X` at the top level are compile errors; they are allowed only for nested classes.
- A member is reachable only if **both** the class and the member are visible. A `public` method in a package-private class is not usable from another package (except through a public supertype or interface).
- **Overriding cannot reduce visibility.** A `public` method in the parent must be `public` in the child. It may widen it (`protected` to `public`).
- A **private constructor** blocks `new` from outside: used by singletons, static factory methods and utility classes (`Math`, `Collections`).
- **`private` methods are not inherited** and so cannot be overridden; a same-named method in a subclass is a new, unrelated method.
- The **module system** (Java 9+) adds a layer above packages: a `public` class in a package that its module does not `exports` is invisible outside the module. Reflection can still bypass `private` through `setAccessible(true)` unless the module system or

a security policy forbids it, so `private` is a design tool, not a security boundary.

Choosing a level. Start with the most restrictive that works (**least privilege**): make fields `private`; make methods `private` unless another class needs them; use package-private for same-package collaboration; use `protected` only for members designed for subclasses; use `public` only for the intended API.

```
public class OrderService {                    // public: the API entry point
    private final OrderRepository repo;        // private: implementation
detail

    public Order place(OrderRequest req) {     // public: the contract
        validate(req);
        return repo.save(toEntity(req));
    }
    private void validate(OrderRequest r) { }              // private helper
    Order toEntity(OrderRequest r) { return null; }        // package-
private: used by tests in same package
    protected void onPlaced(Order o) { }                   // protected: a
hook for subclasses
}
```

3.1.4 Instance Variables and the new Keyword

Instance variables (non-static data members)

An **instance variable** is a field declared inside a class, outside any method, **without `static`**. Every object gets its **own copy**, so the values describe that one object's state.

```
class Employee {
    String name;               // instance variable, default null
    int age;                   // default 0
    double salary = 50_000;    // with an initializer
    boolean active;            // default false
    final String empId;        // blank final: must be assigned exactly once
    static int count;          // NOT an instance variable: one copy for the
whole class

    Employee(String empId) { this.empId = empId; }
}

Employee e1 = new Employee("E1");
Employee e2 = new Employee("E2");
e1.name = "Asha";
System.out.println(e2.name);   // null: e2 has its own copy of 'name'
```

Key properties

Property	Instance variable
Created	When the object is created with `new`
Destroyed	When the object is garbage collected (not before)
Memory	Inside the object, on the heap
Default values	`0`, `0.0`, `false`, `'\u0000'`, `null`, applied automatically
Scope	The whole class (every instance method and constructor), through `this`
Access	All four access modifiers are allowed
Allowed modifiers	`final`, `transient`, `volatile`, plus access modifiers; not `abstract`, `synchronized` or `native`

Ways a value gets assigned (in the order they run)

1. The JVM zeroes every field when the object is allocated (the defaults).
2. Field initializers and instance initializer blocks run, top to bottom, in the order they appear in the source.

3. The constructor body runs and may overwrite anything.

```
class Demo {
    int a = 10;              // 2. initializer
    int b;
    { b = a * 2; }           // 2. instance initializer block runs after 'a =
10' because it appears later
    int c;
    Demo() { c = a + b; }    // 3. constructor
}
// a = 10, b = 20, c = 30
```

Rules to remember

- **Instance members need an object.** A `static` method cannot read an instance variable directly; it has no `this`.

```
class Counter {
    int value;
    static void bad()  { /* value++;  ERROR: non-static variable value cannot
be referenced from a static context */ }
    static void good(Counter c) { c.value++; }     // OK: it has an object to
work on
}
```

- A **`final` instance variable** must be assigned exactly once, either at its declaration, in an instance initializer, or in **every** constructor. Otherwise the code does not compile.
- A local variable or parameter with the same name **shadows** the field; use `this.field` (Section 2.5).
- Fields of **reference type** default to `null`, so using them before assignment gives `NullPointerException`.

How big is an object? An object takes memory for an **object header** plus its fields, rounded up to a multiple of 8 bytes. On a 64-bit HotSpot JVM with compressed class pointers the header is 12 bytes; Java 25 offers *compact object headers* (8 bytes) as an opt-in option, which reduces memory for programs with many small objects. Fields are `4` bytes for `int` / `float`, `8` for `long` / `double`, `1` for `byte` / `boolean`, `2` for `short` / `char`, and `4` (compressed) or `8` for references. The JVM may reorder fields to reduce padding. You can inspect exact layouts with the OpenJDK **JOL** tool.

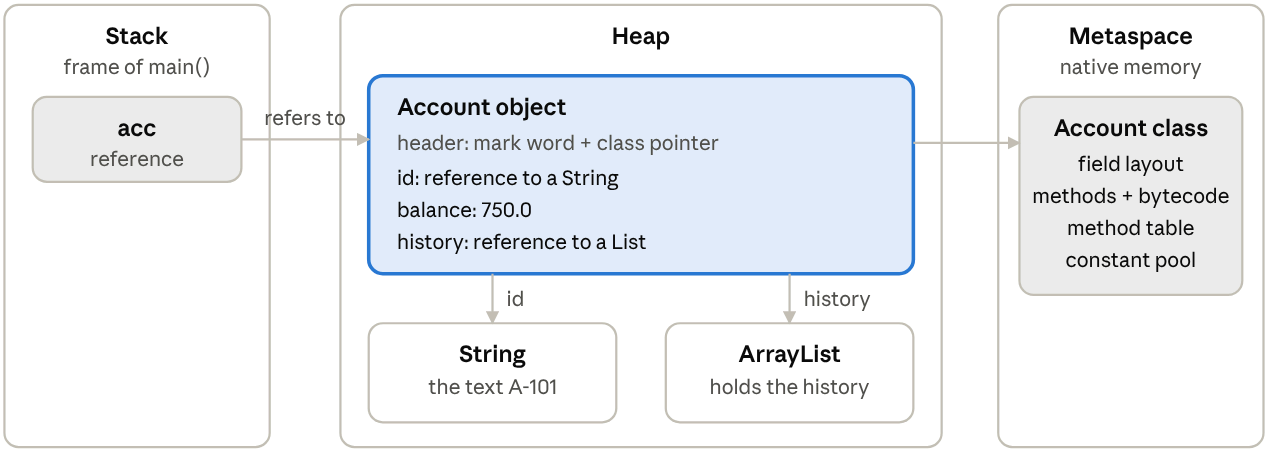
```
An object with one int and one long field:
  header 12 + int 4 + long 8 = 24 bytes
An Integer: header 12 + int 4 = 16 bytes (vs. 4 bytes for a primitive int)
```

The `new` keyword: creating an object

`new` is the operator that **creates an object**: it allocates memory for it, runs a constructor to initialize it, and returns a reference to it.

```
Account acc = new Account("A-101", 500);
//  ^^^^^^^   ^^^   ^^^^^^^^^^^^^^^^^^^
//  type of   new   constructor call
//  the ref
```

One new creates one heap object, linked to shared class data



The heap object holds only this object's own field values. Methods and field layout are shared by every Account object through the class pointer in the header, so a million accounts store the method code once.

Memory after: Account acc = new Account("A-101", 500)

The drawing shows the memory after this statement and a `deposit(250)`: the **reference** sits in the local variable on the stack, the **object** with its own field values sits on the heap, and the header's **class pointer** links to the class's shared metadata in Metaspace.

What happens, step by step

1. **Class loading and initialization (first use only).** If `Account` has not been loaded yet, the ClassLoader loads, links and initializes it (Section 1). Static initializers run once, here.

2. **Memory allocation.** The JVM reserves heap space for the object: header plus all instance fields, including inherited ones. In HotSpot this is normally a very fast *bump-pointer allocation* inside the thread's private **TLAB** (thread-local allocation buffer), so most allocations need no locking.
3. **Default initialization.** All fields are set to their default values (`0`, `false`, `null`). The header is filled in: the class pointer and the identity/lock/age bits.
4. **Constructor call.** The matching constructor runs. It first calls the **parent constructor chain** up to `Object`, then runs field initializers and instance initializer blocks, then the rest of the constructor body (3.1.5 and 3.1.6 give the exact order).
5. **Reference returned.** The expression `new Account(...)` evaluates to a reference, which is stored in `acc`.

In bytecode (`javap -c Demo`):

```
0: new           #7    // class Account            allocate, push the
reference
3: dup                                             duplicate it: one copy
for the constructor
4: ldc           #9    // String A-101
6: ldc2_w        #10   // double 500.0d
9: invokespecial #12   // Method Account."<init>":(Ljava/lang/String;D)V
12: astore_1                                       store the remaining
reference in 'acc'
```

The constructor is compiled to a special method named `<init>`. `new` and `<init>` are separate steps in the bytecode; an object that has been allocated but whose constructor has not finished is **partially constructed**, which is why leaking `this` from a constructor is dangerous (3.1.5).

Facts about `new`

- It **always creates a new object**. It never returns an existing one. Factory methods such as `Integer.valueOf` may return a cached object; `new Integer(5)` (removed from normal use) always allocated.
- It needs a **concrete class** with an **accessible constructor**: you cannot write `new Shape()` for an abstract class or interface (except as an anonymous subclass: `new Runnable() { public void run() {} }`), nor call a private constructor from outside.
- **Arrays** are created with `new` too: `new int[5]`, `new String[3][4]`.
- **Inner classes** need an outer instance: `outer.new Inner()` (3.2).

- The reference can be used directly without storing it: `new Account("A-1", 0).deposit(100);` creates an object that becomes unreachable at once.
- It can fail: `OutOfMemoryError` if the heap is exhausted, `ExceptionInInitializerError` or `NoClassDefFoundError` if class initialization failed, or any exception thrown by the constructor (in which case no reference is returned).
- Objects with no references left are **eligible for garbage collection**; memory is reclaimed by the collector and not by you.
- **Escape analysis** may let the JIT avoid the heap allocation altogether when an object never leaves a method (Section 1). `new` is the semantic model, not a promise of a heap allocation.

Ways to create an object

Way	Calls a constructor?	Notes
`new`	Yes	The normal way
Reflection: `ctor.newInstance(...)`	Yes	Frameworks use this (Spring, Hibernate)
`clone()`	No	Copies the object's fields directly; shallow by default (3.3)
Deserialization	No (for the serializable class)	Rebuilds from bytes; a security-sensitive area
Literals, boxing, lambdas, string concatenation	Handled by the compiler and runtime	`"abc"`, `Integer i = 5`, `() -> {}`

(The JDK internal `Unsafe.allocateInstance` also skips constructors; ordinary code should never use it.)

3.1.5 Constructors, this, super and Destructors

A **constructor** is a special block of code that runs automatically when an object is created with `new`, and its job is to put the new object into a valid initial state.

```
public class Account {
    private final String id;
    private double balance;

    public Account(String id, double opening) {      // constructor: same
name as the class, no return type
        if (opening < 0) throw new IllegalArgumentException("opening balance
cannot be negative");
        this.id = id;
        this.balance = opening;
    }
}
```

Properties of constructors

- The name **must equal the class name**, and there is **no return type**, not even `void`. Adding `void` turns it into an ordinary method that merely shares the class name; the compiler will not complain, and the constructor will silently not run.
- It is called **only through `new`** (or by another constructor with `this(...)` / `super(...)`). You cannot call `acc.Account(...)` on an existing object.
- It can be **overloaded**: several constructors with different parameter lists.
- It may use any **access modifier** (`public` , `protected` , package-private, `private`).
- It cannot be `static` , `final` , `abstract` , `synchronized` , `native` or `strictfp` . (Use a `synchronized (this)` block inside if you must.)
- It is **not inherited** and cannot be overridden. Each class defines its own.
- It can **throw exceptions**. If a constructor throws, no reference is returned and the half-built object is discarded.
- **Interfaces have no constructors** (no state to initialize). **Abstract classes do**, called through `super(...)` from subclasses. **Enum** constructors are implicitly `private` .

	Constructor	Method
Purpose	Initialize a new object	Perform behavior
Name	Class name	Any valid identifier
Return type	None	Required (`void` or a type)

	Constructor	Method
Called by	`new`, `this(...)`, `super(...)`	Explicit call on an object or class
Inherited	No	Yes (unless `private`, or static rules)
Compiler-supplied	Yes: the default constructor	No
Modifiers `static`, `final`, `abstract`	Not allowed	Allowed

The default constructor

If you write **no** constructor, the compiler adds one: a **no-argument constructor with the same access level as the class** that calls `super()`.

```
class Car { String model; }          // you write this
// the compiler produces:
class Car {
    String model;
    Car() { super(); }               // default constructor
}
Car c = new Car();                   // works
```

The moment you declare **any** constructor yourself, the compiler does **not** generate the default one:

```
class Car {
    Car(String model) { }
}
Car c = new Car();                   // COMPILE ERROR: no no-arg constructor
exists
```

This breaks frameworks that need a no-arg constructor (JPA entities, Jackson) unless you add one explicitly. The same rule causes a very common inheritance error:

```
class Parent { Parent(int x) { } }
class Child extends Parent { }                // ERROR: implicit super()
finds no Parent()
class Child2 extends Parent { Child2() { super(10); } }   // OK: explicit
call to the parent constructor
```

Types of constructors

Type	Description	Example
Default	Generated by the compiler when none is written	`Car() { super(); }`
No-argument	Written by you, takes no parameters	`Car() { this.speed = 0; }`
Parameterized	Takes values to initialize fields	`Car(String model, int speed)`
Copy	Takes another object of the same class and copies its state. Java has no automatic copy constructor; you write it	`Car(Car other)`
Private	Blocks `new` from outside; used by singletons, utility classes and factory methods	`private Math() { }`
Canonical and compact	Special forms in records (3.5)	`record P(int x, int y) { P { ... } }`

```
public class Student {
    private String name;
    private int age;
    private final List<String> courses = new ArrayList<>();

    public Student() { }                                    // no-arg
    public Student(String name, int age) {                  // parameterized
        this.name = name;
        this.age = age;
    }
    public Student(Student other) {                         // copy constructor
        this.name = other.name;
        this.age = other.age;
        this.courses.addAll(other.courses);                 // copies the list
contents (a deep-enough copy here)
    }
}
```

Private constructors: three uses

```
// 1. Utility class: no instances ever
public final class StringUtils {
    private StringUtils() { throw new AssertionError(); }
    public static boolean isBlank(String s) { return s == null ||
s.isBlank(); }
}

// 2. Singleton (enum-based is the safest form; this is the classic form)
public class Config {
    private static final Config INSTANCE = new Config();
    private Config() { }
    public static Config getInstance() { return INSTANCE; }
}

// 3. Static factory: names and controls creation
public class Money {
    private final long paise;
    private Money(long paise) { this.paise = paise; }
    public static Money ofRupees(long r) { return new Money(r * 100); }
    public static Money ofPaise(long p)  { return new Money(p); }
}
```

A class with only private constructors cannot be extended by outside classes (a subclass could not call `super(...)`).

Constructor overloading and `this(...)` chaining

To avoid repeating code, one constructor can call another **of the same class** with `this(...)`. The "main" constructor does the real work; the others supply defaults.

```
public class Pizza {
    private final String size;
    private final String crust;
    private final boolean extraCheese;

    public Pizza() { this("medium"); }                        // -> next
    public Pizza(String size) { this(size, "thin"); }         // -> next
    public Pizza(String size, String crust) { this(size, crust, false); }
    public Pizza(String size, String crust, boolean extraCheese) {   // the
real constructor
        this.size = size;
        this.crust = crust;
        this.extraCheese = extraCheese;
    }
}
```

Rules for `this(...)`

- It calls **another constructor of the same class**, chosen by the argument types, and runs it first.
- A constructor can contain `this(...)` **or** `super(...)`, never both (both are "constructor invocations").
- It cannot be recursive: `A() { this(1); } A(int x) { this(); }` is a compile error (recursive constructor invocation).
- Before Java 25, `this(...)` / `super(...)` had to be the **first statement**. See the Java 25 section below.

Too many overloads (the "telescoping constructor" problem) is a design smell; builders and static factories are the usual remedy.

The `this` keyword: all uses

`this` is a reference to **the current object**, the one whose method or constructor is running. It exists only in instance code, not in `static` code.

#	Use	Example
1	Distinguish a field from a parameter or local of the same name	`this.name = name;`
2	Call another constructor of the same class	`this("medium");`
3	Call an instance method of the current object (the `this.` is optional)	`this.validate();`
4	Pass the current object to another method or constructor	`registry.register(this);`
5	Return the current object (fluent, chainable API)	`return this;`
6	Refer to the enclosing instance from an inner class	`Outer.this.value`

```
public class QueryBuilder {
    private final StringBuilder sql = new StringBuilder();
    public QueryBuilder select(String cols) { sql.append("SELECT
").append(cols); return this; }   // 5
    public QueryBuilder from(String table)  { sql.append(" FROM
").append(table); return this; }
    public String build() { return sql.toString(); }
}
String q = new QueryBuilder().select("*").from("users").build();      //
SELECT * FROM users
```

The `super` keyword: all uses

`super` refers to the **parent-class part of the current object**.

#	Use	Example
1	Call a parent constructor	`super(name, age);`
2	Call a parent's version of an overridden method	`super.describe();`

#	Use	Example
3	Access a parent field hidden by a field of the same name	`super.count`
4	Call an interface's default method (qualified form)	`Walkable.super.move();`

```
class Person {
    protected String name;
    Person(String name) { this.name = name; }
    String describe() { return "Person " + name; }
}

class Employee extends Person {
    private final double salary;
    Employee(String name, double salary) {
        super(name);                      // 1: parent constructor, sets
'name'
        this.salary = salary;
    }
    @Override String describe() {
        return super.describe() + " earns " + salary;     // 2: reuse the
parent's logic
    }
}
```

Implicit `super()`. If a constructor does not start with `this(...)` or `super(...)`, the compiler inserts `super();` as its first action. That is why a parent class **must have an accessible no-arg constructor** unless the child calls another one explicitly.

Constructor chaining across the hierarchy. Constructors run **from the top down**: the parent's constructor completes before the child's body starts, all the way up to `Object`.

```
class A { A() { System.out.println("A"); } }
class B extends A { B() { System.out.println("B"); } }
class C extends B { C() { System.out.println("C"); } }

new C();      // prints  A  then  B  then  C
// C() -> implicit super() -> B() -> implicit super() -> A() -> implicit
super() -> Object() ; bodies run Object, A, B, C
```

Rules and traps

- **Never call an overridable method from a constructor.** The parent's constructor runs before the child's fields are initialized, so an override can see default values. The full example is in 3.1.6.
- **Never let `this` escape** during construction: registering `this` as a listener, storing it in a static field, or starting a thread from a constructor exposes a half-built object. The final-field visibility guarantee from the memory model (Multithreading, earlier in this course) holds only if `this` does not escape.
- **Fail fast.** Validate arguments in the constructor (`Objects.requireNonNull`, range checks) so an invalid object can never exist.
- **Constructors and `final` fields.** Every `final` instance field must be assigned in every constructor path (or in an initializer), otherwise the class does not compile.
- A subclass constructor cannot skip the parent's. Even if you do not write `super()`, the call happens.

Java 25: flexible constructor bodies (JEP 513)

Until Java 24, `super(...)` or `this(...)` had to be the **very first statement**, so you could not validate or prepare anything first (the workaround was a static helper inside the call: `super(validate(x))`). Java 25 finalizes **flexible constructor bodies**: you may write statements **before** `super(...)` or `this(...)`, as long as they do **not use the object under construction** (no reading `this` fields, no instance method calls). Two uses:

```
class Employee extends Person {
    private final String officeId;

    Employee(String name, int age, String officeId) {
        // Pre-construction phase (Java 25):
        if (age < 18 || age > 67) throw new IllegalArgumentException("invalid
age");   // validate first
        this.officeId = officeId;      // allowed: initialize a field
declared in THIS class before super(...)
        super(name, age);              // Person's constructor may call an
override that reads officeId; it is already set
    }
}
```

Before Java 25, that override would have seen `officeId == null`, which is exactly the trap in 3.1.6. On Java 21 and earlier, the code above does not compile.

Destructors: why Java has none

C++ has destructors because the programmer frees memory and the destructor runs at a known moment. In Java the **garbage collector** reclaims memory at a time you cannot predict, so there is **no destructor**. You cannot know when (or whether) an object will be collected.

What you do instead depends on what needs cleaning up:

Need	Tool	Notes
Memory	Nothing: garbage collection	Just drop references
Files, sockets, DB connections, locks	**`AutoCloseable` + try-with-resources**	Deterministic: `close()` runs at the end of the block, even on exceptions
Safety net if someone forgets `close()`	**`java.lang.ref.Cleaner`** (Java 9+)	Runs a cleanup action after the object becomes unreachable; no guarantee of timing
Reference-queue based cleanup	`PhantomReference`	The lower-level mechanism Cleaner is built on

```
public class Connection implements AutoCloseable {
    private static final Cleaner CLEANER = Cleaner.create();
    private final Cleaner.Cleanable cleanable;

    public Connection() {
        // The cleanup action must NOT reference 'this', or the object can
never become unreachable
        this.cleanable = CLEANER.register(this, () ->
System.out.println("native handle released"));
    }
    @Override public void close() { cleanable.clean(); }          // explicit
and deterministic
}

try (Connection c = new Connection()) {
    // use it
}   // c.close() is called automatically here
```

`finalize()` is obsolete. It was `Object`'s hook that the collector might call before reclaiming an object. It is deprecated since Java 9, **deprecated for removal since Java 18** (JEP 421), and can be switched off with `--finalization=disabled`. Reasons to avoid it: timing is unpredictable and it may never run; it slows allocation and collection; an exception inside it is silently ignored; it can **resurrect** an object by storing `this` somewhere; and it enables security attacks on partially built objects.

Interview questions on this section

- *Can a constructor be inherited or overridden?* No to both. Subclasses must declare their own and call a parent constructor.
- *Can a constructor be `private`, `protected`, `final`, `static`, `synchronized`?* Private and protected yes; the others no.
- *Can there be a return type on a constructor?* Then it is a normal method, and the class gets the default constructor instead.
- *What if the parent has no no-arg constructor?* The child must call a parent constructor explicitly with `super(args)`.
- *Does an abstract class have a constructor?* Yes; it runs when a concrete subclass is created.
- *Why can't an interface have a constructor?* It has no instance state, and you never create an object of an interface type directly.
- *Is `new` and the constructor the same thing?* No. `new` allocates and then invokes the constructor, which only initializes.

3.1.6 Initialization Order and Pass-by-Value

Two topics that decide whether you can predict what your code does: **in what order Java initializes classes and objects**, and **what really happens when you pass arguments**.

Initialization order

There are two separate initialization events, and they run at different times.

Class initialization happens **once per class**, the first time the class is actively used (first `new`, first call to a static method, first access to a non-constant static field). It runs, in source order, the **static field initializers** and **static blocks**. A superclass is always initialized before its subclass.

Instance initialization happens **every time** `new` creates an object. For one class, in this order:

1. Fields are set to default values (done when the memory is allocated).
2. The constructor is entered and immediately calls the parent constructor (`super(...)`, explicit or implicit), which completes first.
3. When `super(...)` returns, the class's **instance field initializers and instance initializer blocks** run, **in the order they appear in the source**.
4. The rest of the constructor body runs.

For a hierarchy, apply that from the top: Object, then each parent, then the child.

```
class A {
    static { System.out.println("A static block"); }
    { System.out.println("A instance block"); }
    A() { System.out.println("A constructor"); show(); }
    void show() { System.out.println("A.show"); }
}

class B extends A {
    static { System.out.println("B static block"); }
    int x = 5;
    { System.out.println("B instance block, x=" + x); }
    B() { System.out.println("B constructor"); }
    @Override void show() { System.out.println("B.show, x=" + x); }
}

public class Main {
    public static void main(String[] args) {
        System.out.println("main");
        new B();
        System.out.println("--- second object");
        new B();
    }
}
```

Output:

```
main
A static block          <- class init, once, parent first
B static block
A instance block        <- object 1: parent's initializers, then constructor
body
A constructor
B.show, x=0             <- the OVERRIDE runs, but B's field x is not
initialized yet
B instance block, x=5   <- now B's initializers run
B constructor
--- second object
A instance block        <- no static blocks again
A constructor
B.show, x=0
B instance block, x=5
B constructor
```

The key surprise is `B.show, x=0`. `A`'s constructor calls `show()`, and virtual dispatch picks `B`'s override, but `B`'s field initializers have not run yet, so `x` still has its default value. That is why **constructors must not call overridable methods**. (With Java 25 flexible constructor bodies, `B` could assign its own fields *before* calling `super(...)` to avoid this, see 3.1.5.)

Summary of the order

Step	What runs	How often
1	Parent static initializers (fields and blocks, in source order)	Once per class
2	Child static initializers	Once per class
3	Memory allocated, fields zeroed	Per object
4	Child constructor starts, calls parent constructor	Per object
5	Parent instance initializers (fields and blocks), then parent constructor body	Per object
6	Child instance initializers, then child constructor body	Per object

Further rules

- **Static blocks** run once, when the class is initialized. Several static blocks run in textual order. Use them for complex static setup. If one throws, the class initialization fails with `ExceptionInInitializerError`, and every later use gives `NoClassDefFoundError`.
- **Instance initializer blocks** (`{ ... }` without `static`) run for **every object**, before the constructor body but after `super(...)`. They are useful for code shared by all constructors, or in anonymous classes where you cannot write a constructor.
- **With `this(...)` chaining**, instance initializers run **only once**, in the constructor that actually calls `super(...)`, not again in the constructors that delegate through `this(...)`.
- **Textual order matters.** Initializers run in the order written, and a simple-name read of a field before its declaration is a compile error (*illegal forward reference*), though going through `this.x` or a method compiles and returns the default value.

```
class Fwd {
    // int a = b + 1;        // COMPILE ERROR: illegal forward reference
    int b = 10;
    int c = this.d + 1;      // compiles; d is still 0 here, so c = 1
    int d = 5;
}
```

A classic static-order trap

```
class Singleton {
    static Singleton instance = new Singleton();   // runs first: constructor
increments count (0 -> 1)
    static int count = 5;                          // runs second: overwrites
count with 5
    Singleton() { count++; }
}
System.out.println(Singleton.count);               // 5, not 6
```

Because static initializers run top to bottom, the order of declaration changes the result: swap the two lines and the answer becomes `6`.

What triggers class initialization (and what does not)

Triggers it	Does not trigger it
`new Foo()`	Declaring a variable of type `Foo`

Triggers it	Does not trigger it
Calling a static method of `Foo`	Reading a compile-time constant (`static final int X = 5`)
Reading or writing a non-constant static field	Using `Foo.class`
`Class.forName("Foo")`	`ClassLoader.loadClass("Foo")` (only loads)
Initializing a subclass (parent first)	Referring to a subclass's inherited static field through the subclass name (only the declaring class is initialized)
Being the class that contains `main`	

Pass-by-value: how arguments are passed

Java is always pass-by-value. When you call a method, the JVM **copies the value of each argument** into the method's parameter variable. There is no pass-by-reference in Java.

- For a **primitive**, the value is the number itself, so the method works on a copy.
- For an **object**, the value that is copied is the **reference** (the pointer to the object), not the object. The method gets its own copy of the pointer, and both pointers lead to the **same object**.

That explains both behaviors people notice: a method **can modify the object** a passed reference points to, but it **cannot change which object the caller's variable refers to**.

```
static void changePrimitive(int n)          { n = 99; }
// changes a copy
static void mutate(StringBuilder sb)        { sb.append(" world"); }
// modifies the shared object
static void reassign(StringBuilder sb)      { sb = new
StringBuilder("other"); } // only the copy of the reference changes
static void swap(Object a, Object b)        { Object t = a; a = b; b = t; }
// swaps the copies: has no effect outside

public static void main(String[] args) {
    int n = 1;
    changePrimitive(n);
    System.out.println(n);                          // 1
```

```
    StringBuilder s = new StringBuilder("hello");
    mutate(s);
    System.out.println(s);                          // hello world   (object
changed through the copied reference)

    reassign(s);
    System.out.println(s);                          // hello world   (caller's
variable still refers to the same object)

    String x = "A", y = "B";
    swap(x, y);
    System.out.println(x + y);                      // AB  (a swap method
cannot be written in Java)
}
```

Picture of `reassign(s)`:

```
before the call            inside reassign, after the assignment       after
the call
s  ---> [hello world]      s  ---> [hello world]                       s --->
[hello world]
                           sb ---> [other]   (sb was re-pointed)       (sb is
gone; [other] is garbage)
```

The parameter `sb` started as a copy of `s` (same object), and then `sb` was pointed somewhere else. `s` never moved.

Other cases

- **Immutable objects** (`String`, `Integer`, `LocalDate`): the object cannot be changed through any reference, so a method can never alter the caller's value. `void inc(Integer i) { i++; }` creates a new `Integer` for the local copy and leaves the caller's variable alone.
- **Arrays** are objects: a method can change the *elements* of an array passed to it, but `arr = new int[5]` inside the method affects only the local copy.
- **`final` parameters** forbid reassigning the parameter inside the method; they do not make the object immutable.
- **Returning several values** is done with a record, a small class, or an array, not by output parameters: `record MinMax(int min, int max) { }`.

Why "pass-by-reference" is wrong for Java. In a true pass-by-reference language (C++ `void f(int& x)`), assigning to the parameter inside the method changes the caller's variable. Java never does that, which is why a `swap(a, b)` method cannot work. What makes it feel like pass-by-reference is that the *value copied is a reference*, so the **object** is shared.

One-line interview answer: *Java is strictly pass-by-value; for objects, the value passed is a copy of the reference, so the callee can mutate the shared object but cannot rebind the caller's variable.*

Review Questions: Part 3.1

Try every question before reading the answers. Part A is multiple choice, Part B is short answer, Part C is coding and prediction, and Part D is senior level. All answers and explanations are consolidated in the next section.

Part A: Multiple Choice

Q1. Which statement about the Java class hierarchy is true?

- (a) A class may extend several classes
- (b) Every class extends `Object`, directly or indirectly
- (c) Interfaces extend `Object`
- (d) `Object` extends `Class`

Q2. For `class Car { }`, what does the compiler supply?

- (a) `extends Object` and a default constructor `Car() { super(); }`
- (b) Only `extends Object`
- (c) Only a default constructor
- (d) Nothing

Q3. What is printed?

```
Animal a = new Dog();                // Dog extends Animal
System.out.println((a.getClass() == Animal.class) + " " + (a instanceof
Animal));
```

- (a) `true true`
- (b) `false true`

- (c) `true false`
- (d) `false false`

Q4. Which is NOT a way to obtain a `Class` object?

- (a) `Foo.class`
- (b) `obj.getClass()`
- (c) `Class.forName("Foo")`
- (d) `new Class(Foo)`

Q5. Which of these classes are tightly encapsulated?

```
class P { private int a; }
class Q extends P { int b; }
class R { private int c; protected int d; }
class S extends P { private int e; }
```

- (a) P only
- (b) P and S
- (c) P, Q and S
- (d) All four

Q6. `Base` (package `a`) declares `protected void hello()`. In `Sub extends Base` (package `b`), the method has `Base base = new Base(); base.hello();`. What happens?

- (a) Compiles and runs
- (b) Compile error
- (c) Runtime exception
- (d) `NullPointerException`

Q7. Which modifier is allowed on a **top-level** class?

- (a) `private`
- (b) `protected`
- (c) `public`
- (d) `static`

Q8. A parent declares `public void run()`. A child overrides it as `protected void run()`. What happens?

- (a) Compiles; the child is more restrictive, which is fine
- (b) Compile error: an override cannot reduce visibility
- (c) Runtime error
- (d) The child's method is ignored

Q9. What happens?

```
class T {
    int x;
    void f() { int y; System.out.println(x + y); }
}
```

- (a) Prints `0`
- (b) Prints `null`
- (c) Compile error
- (d) Runtime exception

Q10. Which modifier may be applied to a constructor?

- (a) `static`
- (b) `final`
- (c) `private`
- (d) `abstract`

Q11. What is printed?

```
class P { void P() { System.out.println("hi"); } }
new P();
```

- (a) `hi`
- (b) Compile error
- (c) Nothing
- (d) Runtime error

Q12. What happens?

```
class Parent { Parent(int x) { } }
class Child extends Parent { }
```

- (a) Compiles

- (b) Compile error: the implicit `super()` finds no matching constructor
- (c) Runtime error
- (d) Compiles with a warning

Q13. What is printed by `new C();`?

```
class A { A() { System.out.print("A"); } }
class B extends A { B() { System.out.print("B"); } }
class C extends B { C() { System.out.print("C"); } }
```

- (a) `CBA`
- (b) `ABC`
- (c) `C`
- (d) `ACB`

Q14. Can one constructor contain both `this(...)` and `super(...)`?

- (a) Yes, in any order
- (b) Yes, `this` first then `super`
- (c) No, it is a compile error
- (d) Only in abstract classes

Q15. Which statement is FALSE?

- (a) `this` can be used inside a `static` method
- (b) A method can `return this` to allow chaining
- (c) `this(...)` calls another constructor of the same class
- (d) `this` can be passed as an argument

Q16. Which statement about cleanup in Java is correct?

- (a) Java has destructors written as `~ClassName()`
- (b) `finalize()` is the recommended way to release resources
- (c) try-with-resources with `AutoCloseable` gives deterministic cleanup
- (d) `System.gc()` guarantees that `finalize()` runs

Q17. What is printed by `new Q();`?

```
class P {
    P() { System.out.print("P"); show(); }
    void show() { System.out.print("p"); }
}
class Q extends P {
    int v = 7;
    Q() { System.out.print("Q"); }
    @Override void show() { System.out.print(v); }
}
```

- (a) `PpQ`
- (b) `P0Q`
- (c) `P7Q`
- (d) Compile error

Q18. A class has one `static { }` block and one instance `{ }` block. Three objects are created. How many times does each block run?

- (a) Static 1, instance 3
- (b) Static 3, instance 3
- (c) Static 1, instance 1
- (d) Static 3, instance 1

Q19. What does the caller print?

```
static void f(StringBuilder s) { s = new StringBuilder("x"); }
StringBuilder sb = new StringBuilder("a");
f(sb);
System.out.println(sb);
```

- (a) `x`
- (b) `a`
- (c) `null`
- (d) Compile error

Q20. What is `arr[0]` afterwards?

```
static void g(int[] a) { a[0] = 9; a = new int[]{5}; }
int[] arr = {1};
g(arr);
```

- (a) 1
- (b) 5
- (c) 9
- (d) 0

Q21. What is printed?

```
class S {
    static S inst = new S();
    static int count = 5;
    S() { count++; }
}
System.out.println(S.count);
```

- (a) `5`
- (b) `6`
- (c) `0`
- (d) `1`

Q22. Which of these does NOT trigger initialization of class `Foo`?

- (a) `new Foo()`
- (b) `Foo.staticMethod()`
- (c) Reading `Foo.MAX` where `static final int MAX = 10;`
- (d) `Class.forName("Foo")`

Q23. What does the expression `new Account(...)` evaluate to?

- (a) The object itself
- (b) A reference to the new object
- (c) A copy of the object
- (d) The object's size

Q24. If a class does not override `equals`, what does `equals` compare?

- (a) Field values
- (b) Reference identity
- (c) Hash codes only
- (d) Class names

Q25. In Java 25, which statement about constructors is TRUE?

- (a) `super(...)` must still be the first statement
- (b) Statements that do not use `this` may appear before `super(...)`
- (c) A constructor may call `this(...)` twice
- (d) Constructors may be declared `final`

Part B: Short Answer

Q26. Explain the single-root class hierarchy. Why does `Object` exist, and which methods does it declare?

Q27. Distinguish a class from an object using the terms state, behavior and identity.

Q28. Differentiate data hiding, encapsulation and abstraction.

Q29. Why are getters and setters alone not real encapsulation? Show how a getter can leak mutable state and fix it.

Q30. List the four access levels with what each allows, and explain the special rule for `protected` across packages.

Q31. List the steps the JVM performs for `new Account("A-1", 100)`.

Q32. Compare a constructor and a method. When does the compiler generate a default constructor, and when does it not?

Q33. List all uses of `this` and all uses of `super`.

Q34. Why does Java have no destructor? What do you use instead?

Q35. Explain why Java is pass-by-value, and why a `swap(a, b)` method cannot work.

Part C: Coding and Prediction

C1. Write a fully encapsulated `Temperature` class: a private Celsius value, a constructor that rejects values below -273.15, a getter for Celsius, a computed `getFahrenheit()`, and a `raise(double delta)` behavior method. No plain setter.

C2. This class leaks its internal state. Fix it:

```
class Roster {
    private final List<String> names = new ArrayList<>();
    private final Date created;
    Roster(Date created) { this.created = created; }
    List<String> getNames() { return names; }
    Date getCreated() { return created; }
}
```

C3. Write a `Pizza` class with fields `size`, `crust`, `extraCheese`, three overloaded constructors chained with `this(...)`, and a copy constructor.

C4. Predict the exact output of `new Derived();`:

```
class Base {
    static int sb = init("Base.static field");
    int ib = init("Base.instance field");
    static { System.out.println("Base.static block"); }
    { System.out.println("Base.instance block"); }
    Base() { System.out.println("Base.constructor"); }
    static int init(String s) { System.out.println(s); return 1; }
}
class Derived extends Base {
    static int sd = init("Derived.static field");
    int id = init("Derived.instance field");
    static { System.out.println("Derived.static block"); }
    { System.out.println("Derived.instance block"); }
    Derived() { this(10); System.out.println("Derived.constructor()"); }
    Derived(int x) { System.out.println("Derived.constructor(int)"); }
}
```

C5. Write a utility class `Validators` that cannot be instantiated or extended, with a static method `requirePositive(int n)`. Then write a lazy, thread-safe singleton `Registry` using the initialization-on-demand holder idiom.

C6. Predict the output:

```
class R implements AutoCloseable {
    final String n;
    R(String n) { this.n = n; System.out.println("open " + n); }
    public void close() { System.out.println("close " + n); }
}
try (R a = new R("A"); R b = new R("B")) {
    System.out.println("body");
    throw new RuntimeException("x");
} catch (Exception e) {
    System.out.println("catch " + e.getMessage());
} finally {
    System.out.println("finally");
}
```

C7. Write a method `static void printHierarchy(Object o)` that prints the class name of `o` and every superclass up to `Object`, then the interfaces directly implemented by `o`'s class.

Part D: Senior Level

S1. Why did Java choose single inheritance of classes and a single-root hierarchy? What does each decision buy, and what does it cost?

S2. Describe precisely what happens in the JVM for `new Foo()`, including class initialization, allocation (TLAB), header setup, constructor chain and the bytecode. What is a partially constructed object, and why can leaking `this` from a constructor be dangerous?

S3. Why is calling an overridable method from a constructor dangerous? Give three different ways to fix or avoid it, including a Java 25 option.

S4. Explain how class initialization can deadlock, and what `ExceptionInInitializerError` followed by `NoClassDefFoundError` means.

S5. Compare `finalize()`, `Cleaner`, `PhantomReference` and try-with-resources. Why must a `Cleaner` action not capture `this`?

S6. Beyond private fields, which Java features support encapsulation (records, sealed types, modules, `final`, nested classes)? When is a public field acceptable?

S7. Why does a subclass in another package fail to call a protected method through a parent-typed reference, but succeed through its own type? What design reason sits behind this rule?

S8. What does it mean that a class's identity is its name plus its ClassLoader? Describe a `ClassCastException` with identical class names, and relate it to application servers.

S9. Estimate the heap size of an instance of `class P { int a; long b; byte c; Object d; }` on a 64-bit HotSpot JVM with compressed references. How would you verify it, and what changes with compact object headers in Java 25?

S10. A colleague writes `static void swap(Integer a, Integer b) { Integer t = a; a = b; b = t; }` and is surprised it does nothing. Explain, and give three correct designs for returning two results.

S11. A service class has a public constructor with eight parameters. Compare telescoping constructors, a static factory, a builder and a record as remedies, and say when you would choose each.

S12. Predict and explain the output, then say what you would change:

```
class Config {
    static final Config DEFAULT = new Config();
    static final String PREFIX = compute();
    final String name;
    Config() { name = PREFIX + "default"; }
    static String compute() { return "cfg-"; }
}
System.out.println(Config.DEFAULT.name);
```

Answers: Part 3.1

Part A: Multiple Choice

Q	Answer	Explanation
1	(b)	Single-root hierarchy. Classes have one superclass; interfaces are outside the class tree.
2	(a)	The compiler adds `extends Object` and, when no constructor is written, `Car() { super(); }`.
3	(b) `false true`	`getClass()` returns the real runtime class (`Dog`), so it is not `Animal.class`; `instanceof` accepts subtypes.

Q	Answer	Explanation
4	(d)	`Class` has no public constructor; the JVM creates these objects.
5	(b)	Tightly encapsulated means every field is `private`, in the class and in every parent. Q has a package-private field, R has a `protected` field.
6	(b)	Outside the package, `protected` is reachable only through a reference of the subclass type, not through the parent type.
7	(c)	Top-level types are `public` or package-private only.
8	(b)	An override cannot reduce visibility. It may widen it.
9	(c)	The local `y` has no default value; reading it before assignment is a compile error.
10	(c)	Constructors can be `private`, but not `static`, `final` or `abstract`.
11	(c)	`void P()` is an ordinary method. The compiler supplied the default constructor, which prints nothing.
12	(b)	`Child` gets a default constructor that calls `super()`, and `Parent` has no no-arg constructor.
13	(b) `ABC`	Constructors run top-down: parent bodies finish first.
14	(c)	A constructor may start with `this(...)` or `super(...)`, never both.
15	(a)	`this` does not exist in static code, because no object is involved.
16	(c)	Java has no destructors; `finalize()` is deprecated for removal; `System.gc()` is only a hint.

Q	Answer	Explanation
17	(b) `P0Q`	`P` prints `P`, then the override reads `v` before `Q`'s initializer ran (`0`), then `Q` prints.
18	(a)	Static blocks run once per class; instance blocks run for each object.
19	(b)	`s` is a copy of the reference; re-pointing it does not affect `sb`.
20	(c)	The element change goes through the shared array; the reassignment affects only the local reference.
21	(a) `5`	`inst` is created first (`count` 0 to 1), then `count = 5` overwrites it.
22	(c)	A compile-time constant is inlined, so reading it does not initialize the class.
23	(b)	`new` yields a reference; the object itself lives on the heap.
24	(b)	The default `Object.equals` is the same as `==`.
25	(b)	JEP 513 (final in Java 25): statements that do not use the object under construction may precede `super(...)` or `this(...)`.

Part B: Short Answer (model answers)

Q26. Every class has exactly one superclass, and the chain always ends at `java.lang.Object`, so `Object` is the root of all classes. It exists to give every object common behavior and to give code a universal type. It declares `getClass`, `hashCode`, `equals`, `toString`, `clone`, `finalize`, `wait` (three overloads), `notify` and `notifyAll`.

Q27. A class is the blueprint (a type) loaded once; an object is a runtime instance created with `new`. An object has **state** (its field values), **behavior** (its methods, shared with all objects of the class) and **identity** (it is a distinct object even if its state equals another's; compared with `==`).

Q28. Data hiding means outsiders cannot touch data directly (private fields). Encapsulation is the packaging of data and behavior into a class with controlled access, using data hiding as its tool. Abstraction means exposing *what* a type does and hiding *how*, typically through interfaces and abstract classes.

Q29. A generated getter/setter pair for every field exposes the state just as a public field would, and callers then make the decisions that the object should make. Better: expose behavior (`withdraw(100)` instead of `setBalance(getBalance() - 100)`). A getter leaks state when it returns a mutable internal object, such as the list in `getNames()` , because callers can change it. Fix by returning an unmodifiable view or an immutable copy (`List.copyOf`), and by copying mutable inputs on the way in.

Q30. `public` : everywhere. `protected` : same package, plus subclasses in other packages. Package-private (no keyword): same package only. `private` : only inside the declaring class (and its nested classes). The `protected` rule: outside the package, a subclass may use the member only through a reference whose type is that subclass or its subtypes, not through a reference of the parent type.

Q31. (1) Load, link and initialize `Account` if this is its first use. (2) Allocate heap memory for the header and all fields (usually a bump-pointer allocation in the thread's TLAB). (3) Zero the fields and fill in the header (class pointer, lock and hash bits). (4) Invoke the constructor: `super(...)` chain up to `Object` , then instance initializers and field initializers, then the constructor body. (5) The expression yields the reference, which is stored in the variable.

Q32. A constructor has the class name, no return type, is called by `new` / `this(...)` / `super(...)` , is not inherited, and cannot be `static` , `final` or `abstract` . A method has any name and a return type and is inherited. The compiler generates a no-arg default constructor (same access as the class, calling `super()`) only when the class declares **no** constructor at all. Declare any constructor and the default disappears.

Q33. `this` : (1) disambiguate field vs parameter, (2) call another constructor `this(...)` , (3) call an instance method of the current object, (4) pass the current object as an argument, (5) return the current object for chaining, (6) `Outer.this` for the enclosing instance. `super` : (1) call a parent constructor `super(...)` , (2) call the parent's version of an overridden method `super.m()` , (3) access a hidden parent field `super.f` , (4) call an interface default method `Iface.super.m()` .

Q34. Memory is reclaimed by the garbage collector at times the program cannot predict, so there is no moment at which a destructor could run reliably. `finalize()` was the substitute but is deprecated for removal. Use **try-with-resources and `AutoCloseable`** for deterministic cleanup, `Cleaner` as a safety net, and plain reference-dropping for memory.

Q35. A call copies each argument's value into the parameter. For a primitive the copy is the number; for an object it is a copy of the **reference**. The callee can modify the shared object through its copy, but re-pointing its parameter does not change the caller's variable. A `swap(a, b)` method swaps only its own two local copies, so the caller sees no change.

Part C: Coding and Prediction (model solutions)

C1. Temperature

```
public final class Temperature {
    private static final double ABSOLUTE_ZERO = -273.15;
    private double celsius;

    public Temperature(double celsius) { this.celsius = require(celsius); }

    public double getCelsius()    { return celsius; }
    public double getFahrenheit() { return celsius * 9 / 5 + 32; }       //
derived, not stored

    public void raise(double delta) { celsius = require(celsius + delta); }
// behavior, not a setter

    private static double require(double c) {
        if (Double.isNaN(c) || c < ABSOLUTE_ZERO)
            throw new IllegalArgumentException("below absolute zero: " + c);
        return c;
    }
}
```

C2. Fixed `Roster`

```
class Roster {
    private final List<String> names = new ArrayList<>();
    private final Date created;

    Roster(Date created) { this.created = new Date(created.getTime()); }
// defensive copy in

    void add(String name)      { names.add(Objects.requireNonNull(name)); }
// controlled mutation
    List<String> getNames()    { return List.copyOf(names); }
// immutable snapshot out
    Date getCreated()          { return new Date(created.getTime()); }
// defensive copy out
}
```

Using `java.time.Instant` instead of `Date` would remove the need for copies, because `Instant` is immutable.

C3. Pizza

```
public class Pizza {
    private final String size;
    private final String crust;
    private final boolean extraCheese;

    public Pizza()                    { this("medium"); }
    public Pizza(String size)         { this(size, "thin", false); }
    public Pizza(String size, String crust, boolean extraCheese) {     // the
real constructor
        this.size = Objects.requireNonNull(size);
        this.crust = Objects.requireNonNull(crust);
        this.extraCheese = extraCheese;
    }
    public Pizza(Pizza other)         { this(other.size, other.crust,
other.extraCheese); }   // copy constructor
}
```

C4. Output

```
Base.static field
Base.static block
Derived.static field
Derived.static block
Base.instance field
Base.instance block
Base.constructor
Derived.instance field
Derived.instance block
Derived.constructor(int)
Derived.constructor()
```

Static initialization runs parent first, in source order. Then `Derived()` delegates with `this(10)` to `Derived(int)`, which implicitly calls `super()`. The parent's instance initializers and constructor run, then `Derived`'s instance initializers run **once**, in `Derived(int)` (the constructor that calls `super`), then its body. Finally control returns to the body of `Derived()`.

C5. Utility class and holder singleton

```
public final class Validators {                        // final: cannot be
extended
    private Validators() { throw new AssertionError("no instances"); }   //
private: cannot be instantiated
    public static int requirePositive(int n) {
        if (n <= 0) throw new IllegalArgumentException("must be positive: " +
n);
        return n;
    }
}

public final class Registry {
    private Registry() { }
    private static final class Holder {                // loaded and
initialized only on first use
        static final Registry INSTANCE = new Registry();
    }
    public static Registry getInstance() { return Holder.INSTANCE; }
}
```

The holder idiom is lazy and thread-safe without locks, because the JVM guarantees that class initialization happens once and is synchronized (Section 1 and the Multithreading notes).

C6. Output

```
open A
open B
body
close B
close A
catch x
finally
```

Resources are closed in **reverse order of opening**, before the `catch` and `finally` blocks run.

C7. Hierarchy printer

```
static void printHierarchy(Object o) {
    Class<?> c = o.getClass();
    for (Class<?> k = c; k != null; k = k.getSuperclass()) {
        System.out.println(k.getName());
    }
    for (Class<?> i : c.getInterfaces()) {
        System.out.println("implements " + i.getName());
    }
}
// printHierarchy(new ArrayList<String>()):
// java.util.ArrayList / java.util.AbstractList /
java.util.AbstractCollection / java.lang.Object
// implements java.util.List / java.util.RandomAccess / java.lang.Cloneable /
java.io.Serializable
```

Part D: Senior Level (model answers)

S1. *Single inheritance of classes* avoids the **diamond problem for state**: with two parent classes you would have two copies of inherited fields or ambiguous constructor and field resolution (the complexity of C++ virtual bases). It keeps object layout simple (a field has one fixed offset along one chain) and virtual dispatch cheap. Multiple *type* inheritance is still available through interfaces, and since Java 8 default methods allow multiple inherited behavior with explicit conflict resolution. A *single root* gives a universal type,

uniform methods for every object (`toString` , `equals` , `hashCode` , monitors), and made generic containers and reflection possible. The costs: you cannot reuse implementation from two parents, so you use composition and delegation; every object pays for `Object` 's baggage (header bits for locking and hash, default `equals` / `hashCode` that are often wrong for value types); and primitives sit outside the hierarchy, which causes boxing.

S2. On first use the ClassLoader loads, links and initializes the class (static initializers run). The JVM then allocates the object, usually by bumping a pointer in the thread's TLAB with no locking, falling back to a slower path when the TLAB is full. The memory is zeroed, the header is written (class pointer, mark word), and the constructor `<init>` runs: the `super(...)` chain to `Object` , instance initializers, constructor body. Bytecode is `new` , `dup` , argument pushes, `invokespecial <init>` , then a store. Between allocation and the end of the constructor the object is **partially constructed**. If `this` escapes (stored in a static field, registered as a listener, passed to another thread, or an overridable method is called), other code can observe default values or a half-built invariant. In concurrent code this also voids the final-field guarantee of the Java Memory Model, which only holds if `this` does not escape the constructor.

S3. The parent constructor runs before the child's field initializers, so a virtual call from it dispatches to the child's override, which can read uninitialized fields (the `P0Q` example). Remedies: (1) call only `private` , `final` or `static` methods from constructors, so there is no dynamic dispatch; (2) make the class `final` or the hook method `final` ; (3) move work out of the constructor into a static factory or builder that completes construction before any hook runs; (4) in a hierarchy you control, add an explicit `init()` called by the factory after construction (two-phase construction, at the price of a weaker invariant); (5) with Java 25 flexible constructor bodies, initialize the subclass's own fields **before** `super(...)` , so the override sees valid values.

S4. Class initialization runs under a per-class initialization lock. If thread 1 initializes class `A` whose static initializer touches class `B` , while thread 2 initializes `B` whose static initializer touches `A` , each holds one lock and waits for the other: a deadlock that shows up in a thread dump as two threads inside `<clinit>` . Avoid cyclic static dependencies, keep static initializers small, never start threads or block in them, and use the holder idiom for lazy singletons. If a static initializer throws, the JVM wraps the failure in `ExceptionInInitializerError` and marks the class unusable. Every later use throws `NoClassDefFoundError: Could not initialize class X` , **without** the original cause, so the real exception is in the first log entry, not the later ones.

S5.

Mechanism	Timing	Status
try-with-resources / `AutoCloseable`	Deterministic, at the end of the block	The primary tool
`Cleaner`	After the object becomes unreachable; no guarantee when or whether	Safety net for native or off-heap resources
`PhantomReference` + `ReferenceQueue`	After the collector decides the object is unreachable	Low-level building block, used by `Cleaner`
`finalize()`	Unpredictable, may never run	Deprecated for removal since Java 18

A `Cleaner` action that captures `this` (directly, or through a lambda or inner class) keeps the object strongly reachable from the cleaner's own list, so it is never considered unreachable and the cleanup never runs: a memory leak. The action should be a `static` nested class holding only the resource state.

S6. `final` (fields, methods, classes) prevents change; **records** make immutable value carriers whose state is exposed but cannot be mutated; **sealed** types control who may extend; **package-private** access and **private nested classes** hide implementation types; **interfaces** expose behavior instead of structure; **static factories** hide concrete classes; and **modules** (`exports`) hide whole packages even if their classes are `public` . A public field is acceptable for constants (`public static final` immutable values), for immutable value holders inside an internal package, or for a private nested helper where no outside code can depend on it. It is not acceptable for any mutable state that is part of a published API.

S7. `protected` lets a subclass use inherited members **of its own instances**. If it also worked through a parent-typed reference, any subclass, in any package, could reach into the protected state of *any* `Base` object, including instances of other, unrelated subclasses, which would defeat the point of protection. The JLS therefore permits access outside the package only when the qualifying expression's type is the subclass itself (or a subtype). Through the parent-typed reference `base.hello()` is rejected for the same reason that a sibling subclass's instance is.

S8. At runtime a type is identified by its fully qualified name **and** its defining ClassLoader. If two loaders each load their own copy of `com.app.User` , they are two different types: passing an instance from one world to the other and casting produces a

`ClassCastException` whose message names the same class twice (newer JDKs also name the loaders). This happens in application servers, where each web app has its own loader, and after a hot redeploy, when old and new loaders coexist. It also means a static singleton exists once per loader. Sharing across loaders requires the shared types (usually interfaces) to be loaded by a common parent loader.

S9. Layout with a 12-byte header: `int a` at 12 (4 bytes), `long b` at 16 (8), `byte c` and the 4-byte reference `d` packed after, for about **29 bytes of content, rounded up to a multiple of 8, so 32 bytes**. The JVM may order fields differently but the total comes out the same here. Verify with the OpenJDK **JOL** tool (`ClassLayout.parseClass(P.class).toPrintable()`). Java 25's compact object headers (opt-in with `-XX:+UseCompactObjectHeaders`) cut the header to 8 bytes; for this particular class the padding to 8 bytes still gives 32, so savings show up only for shapes where the 4 saved bytes cross an alignment boundary, which is common in programs with many small objects.

S10. `Integer` is immutable and Java passes copies of references, so the method swaps only its two local variables; the caller's variables still refer to their original objects. Correct designs: (1) return both values in a record, `record Pair<A, B>(A first, B second)`, and let the caller destructure it; (2) put the values in a container and swap inside it (`Collections.swap(list, i, j)`, or swapping array elements); (3) emulate an output parameter with a small mutable holder (`AtomicReference`, `int[1]`), which works but is usually a design smell. Prefer (1) or (2).

S11. *Telescoping constructors* do not scale: arguments are positional and easy to swap. A *static factory* gives names and caching, but still needs many parameters. A *builder* reads clearly, handles many optional values, and validates in `build()`; the cost is extra code and a mutable builder object. A *record* gives a concise immutable value with a compact constructor for validation, but all components are positional, so combine it with a builder when there are many. Rules of thumb: up to about four required values, a plain constructor; many optional settings, a builder; a fixed set of immutable value components, a record; named or cached creation, a static factory. In Spring code, group related parameters into a configuration object and use constructor injection.

S12. Output: **`nulldefault`**. `DEFAULT` is initialized first (textual order), and its constructor reads `PREFIX`, which has not been assigned yet. `PREFIX` is not a compile-time constant because it is initialized by a method call, so it is not inlined and still holds `null`, giving `"null" + "default"`. Fixes: declare `PREFIX` **before** `DEFAULT`; or make it a true constant

(`static final String PREFIX = "cfg-";`, which `javac` inlines); or build `DEFAULT` lazily with the holder idiom. In general, avoid static fields whose initializers depend on one another's order.

End of Part 3.1. Next is **Part 3.2: static, final, abstract and nested classes**. Tell me when to start, or flag anything here you want expanded first.